

El Plan de Gestión de Configuraciones en la era DevOps

En la SCM clásica (Software Configuration Management), el **Plan de Gestión de Configuraciones (PGC)** era un documento Word que describía procedimientos, roles, líneas base, control de cambios y auditorías. En DevOps, ese plan se **materializa como código** (*Configuration as Code*): vive dentro del repositorio en archivos de texto versionados, revisables por *pull request* y ejecutables por máquinas.

1. `.gitignore` — Definición del alcance del ítem de configuración

Función: Especifica qué archivos y directorios **no** deben formar parte del repositorio (binarios compilados, secretos, dependencias descargables, archivos de IDE, logs, etc.).

Cómo garantiza las propiedades SCM:

Propiedad	Explicación
Integridad	Evita que artefactos generados o efímeros contaminen la línea base. Solo se versiona la *fuente de verdad*; lo demás se reconstruye. También previene la fuga accidental de secretos (<code>.env</code> , <code>*.pem</code>) que romperían la integridad del ítem.
Reproducibilidad	Al excluir dependencias locales (ej. <code>node_modules/</code> , <code>target/</code>), obliga a que la construcción se base en manifiestos declarativos (<code>package.json</code> , <code>pom.xml</code>), garantizando que cualquier entorno obtenga lo mismo desde cero.

Control

Define explícitamente el **perímetro del ítem de configuración**. Es la versión moderna de la "lista de ítems de configuración" del PGC clásico: lo que está dentro es controlado; lo que está fuera, no.

2. `Dockerfile` — Empaquetado reproducible del entorno de ejecución

Función: Declara, paso a paso, cómo construir una imagen inmutable que contiene la aplicación y todas sus dependencias de sistema operativo y runtime.

Cómo garantiza las propiedades SCM:

Propiedad	Explicación
Integridad	Al fijar versiones (FROM python:3.12.3-slim , RUN pip install -r requirements.txt) y usar *digests* SHA256, se asegura que la imagen construida corresponde exactamente al estado del código y sus dependencias. El hash de la imagen es un identificador verificable del ítem de configuración.
Reproducibilidad	Elimina el clásico "en mi máquina funciona": el mismo Dockerfile produce la misma imagen en cualquier host. Las capas y el orden de instrucciones determinan una caché determinista, y con --no-cache se reconstruye idénticamente.
Control	Todo cambio en el entorno (versión de

librería, variable, puerto) pasa por un `*commit*` y `*pull request*`. El Dockerfile es la **línea base del entorno de ejecución**, auditable y reversible mediante `git revert`.

3. `.github/workflows/ci.yml` — Automatización del control de cambios

Función: Define el pipeline de Integración Continua (GitHub Actions): cuándo se ejecuta (push, PR), qué pasos corre (lint, tests, build, escaneo de seguridad) y bajo qué condiciones se aprueba o rechaza un cambio.

Cómo garantiza las propiedades SCM:

Propiedad	Explicación
Integridad	Cada <code>*push*</code> o PR dispara validaciones automáticas (tests unitarios, análisis estático, SAST). Ningún cambio entra a la línea base si no pasa las compuertas de calidad, actuando como la auditoría de configuración automatizada del PGC clásico.
Reproducibilidad	El workflow se ejecuta en entornos limpios y efímeros definidos como código. Cualquier persona puede reproducir localmente los mismos pasos (con <code>act</code> , por ejemplo) y obtener resultados equivalentes.
Control	Formaliza el proceso de control de cambios : quién puede aprobar, qué ramas protege, qué artefactos se publican. Reemplaza los formularios de

solicitud de cambio (RFC) por reglas ejecutables y trazables en el historial de `*runs*`.

4. ``CHANGELOG.md`` (o ``version.txt`` / ``semantic-release` config`) — Trazabilidad de la línea base

Función: Documenta de forma estructurada y versionada los cambios relevantes entre releases (nuevas features, correcciones, breaking changes), alineado con Semantic Versioning.

Cómo garantiza las propiedades SCM:

Propiedad	Explicación
Integridad	Cada versión publicada tiene un registro inmutable de qué cambió respecto a la anterior. Permite verificar que el artefacto desplegado corresponde a los cambios declarados.
Reproducibilidad	Al conocer la versión exacta y su changelog, cualquier equipo puede reconstruir el estado del sistema en un punto histórico (combinado con tags de Git).
Control	Es el registro de auditoría moderno: responde a "¿qué cambió, cuándo, quién y por qué?" sin necesidad de un documento Word separado. Los tags de Git (v1.4.2) actúan como líneas base inmutables.

5: ``.editorconfig`` / ``renovate.json`` / ``SECURITY.md``

Para redondear el mapa, otros archivos que materializan partes del PGC:

- ``.editorconfig`` → normaliza estilos entre editores: **integridad** del formato del código fuente.
- ``.renovate.json`` / ``.dependabot.yml`` → política automatizada de actualización de dependencias: **control** de cambios en la cadena de suministro.
- ``.SECURITY.md`` → política de reporte de vulnerabilidades: parte del **plan de aseguramiento** del ítem de configuración.

Síntesis: del documento Word al repositorio

Elemento del PGC clásico	Equivalente DevOps
Lista de ítems de configuración	<code>.gitignore</code> , estructura del repo
Procedimiento de construcción	<code>Dockerfile</code> , <code>Makefile</code>
Control de cambios / CCB	<code>ci.yml</code> , branch protection, PR reviews
Auditoría de configuración	Tests automáticos, SAST, <code>CHANGELOG.md</code>
Líneas base	Tags de Git, imágenes con digest, releases
Roles y responsabilidades	<code>CODEOWNERS</code> , políticas de merge

En DevOps, el Plan de Gestión de Configuraciones deja de ser un documento estático y pasa a ser un conjunto de archivos ejecutables y versionados que garantizan integridad (nada se contamina ni se pierde), reproducibilidad (cualquiera reconstruye el mismo estado) y control (todo cambio es explícito, revisable y reversible). El repositorio mismo se convierte en el PGC vivo.